

Architectural Note: The Role of `__init__.py`

The presence of `__init__.py` (as shown in the repository image) designates the directory as a standard Python package. This allows external scripts to import `parser.py` using dot notation (e.g., `from package import parser`). It establishes the package namespace, ensures controlled module loading, and allows initialization logic to execute when the directory is first imported.

TARGET FILE	TOTAL LINES	LANGUAGE	DOMAIN
parser.py	185 Lines	Python 3.x	Code Analysis / AST

#	SOURCE CODE	TECHNICAL EXPLANATION
1-2	<code>import ast</code> <code>import os</code>	Imports standard libraries required for Abstract Syntax Tree (AST) parsing and operating system level file/path manipulations.
4-13	<code>def parse_python_file(file_path):</code> <code>"""</code> <code>Parse a Python file and return its</code> <code>basic code structure... """</code>	Defines the primary parsing routine. The docstring notes it extracts structural dependencies (classes, methods, imports) without executing the target file.
15-17	<code>if not file_path.endswith(".py"):</code> <code>return None</code>	Guard clause ensuring that the parsing engine only attempts to process valid Python source files. Bypasses non-Python extensions.
19-24	<code>try: with open(file_path, "r",</code> <code>encoding="utf-8") as f: code =</code> <code>f.read() tree = ast.parse(code,</code> <code>filename=file_path)</code>	Safely loads the file content using UTF-8 encoding. Uses <code>ast.parse()</code> to parse the raw code string into a traversable syntax tree representation.
26-30	<code>structure = { "functions": [],</code> <code>"classes": [], "imports": [], }</code>	Initializes a dictionary schema that will aggregate and categorize all structural elements discovered during the AST traversal.
32-40	<code>for node in ast.walk(tree): if</code> <code>isinstance(node, (ast.FunctionDef,</code> <code>ast.AsyncFunctionDef)):</code> <code>structure["functions"].append({</code> <code>"name": node.name, "line":</code> <code>node.lineno, "parent_class": None, })</code>	Begins walking the syntax tree. Identifies both standard and asynchronous function definitions. Records the function's name and starting line number, defaulting its parent class relationship to None.

#	SOURCE CODE (CONT.)	TECHNICAL EXPLANATION
42-46	<pre>elif isinstance(node, ast.ClassDef): structure["classes"].append({ "name": node.name, "line": node.lineno, })</pre>	Continues AST walk matching. Detects standard Python class definitions and catalogs their identifying name and line number within the file.
48-51	<pre>elif isinstance(node, ast.Import): for alias in node.names: structure["imports"].append(alias.name)</pre>	Extracts standard module imports (e.g., <code>import os, sys</code>). Iterates over possible multiple aliases to build an accurate file dependency map.
53-56	<pre>elif isinstance(node, ast.ImportFrom): if node.module: structure["imports"].append(node.module)</pre>	Extracts specific imports (e.g., <code>from module import func</code>). Captures the base module name to understand the broader package dependencies.
58-71	<pre>for node in ast.walk(tree): if isinstance(node, ast.ClassDef): for child in node.body: if isinstance(child, (ast.FunctionDef...)): # Matches logic block function["parent_class"] = node.name</pre>	Executes a secondary AST pass to accurately establish parent-child relationships for methods. Cross-references function line numbers within class bodies to map methods to their parent classes.
73-80	<pre>return structure except (SyntaxError, UnicodeDecodeError, OSError): return None</pre>	Returns the parsed metadata. Gracefully catches file permission errors, encoding issues, or corrupted syntax, returning <code>None</code> to prevent halting the broader codebase scan.
82-97	<pre>def deep_codebase_search(target_dir, search_term): """ Search a codebase for: ... """ results = [] search_lower = search_term.lower()</pre>	Initializes the secondary utility: a deep recursive text and structural search. Normalizes the search term to lowercase to guarantee case-insensitive matching across the codebase.
99-109	<pre>for root, _, files in os.walk(target_dir): for file in files: file_path = os.path.join(root, file) if search_lower in file.lower(): results.append({...})</pre>	Traverses the full directory tree recursively. Appends a result if the user's search term matches any part of the physical filename.

#	SOURCE CODE (CONT.)	TECHNICAL EXPLANATION
111 -13 5	<pre> if file.endswith(".py"): parsed = parse_python_file(file_path) if parsed: for func in parsed["functions"]: if search_lower in func["name"].lower(): parent = func.get("parent_class") # Formatting logic... results.append({...}) </pre>	For Python files, invokes the AST parser. Scans the extracted function list for name matches. If a match is found, dynamically formats the output to distinguish between standalone functions and class-bound methods.
137 -14 6	<pre> for cls in parsed["classes"]: if search_lower in cls["name"].lower(): results.append({ "type": "class", "path": file_path, "details": f"Class '{cls['name']}'..." }) </pre>	Iterates over extracted class definitions. Evaluates class names against the search term, cataloging any matches alongside the file path and specific line number of the definition.
148 -15 6	<pre> for imported_module in parsed["imports"]: if search_lower in imported_module.lower(): results.append({ "type": "import", "path": file_path, "details": f"Import '{imported_module}'" }) </pre>	Evaluates all internal and external dependencies loaded in the file. Identifies if the target search term exists within the imported module paths.
158 -18 3	<pre> try: with open(file_path, "r", encoding="utf-8") as f: source = f.read() if search_lower in source.lower(): results.append({ "type": "code", "path": file_path, "details": "Code content match", }) except (UnicodeDecodeError, OSError): continue </pre>	Acts as a fallback plain-text search on the raw file content. This catches inline strings, variable names, and logic paths not captured by structural AST nodes. Safely ignores files that fail to read.
184 -18 5	<pre> return results </pre>	Finalizes the deep_codebase_search routine by returning the aggregated list of dictionaries containing all filename, structural, and inline textual matches.